

PARALLEL COMPUTING — CLASS 0 DIAGNOSTIC

Prerequisite & problem-solving diagnostic • 2 hours

Purpose: This session checks your current programming, algorithmic, data, and systems knowledge. You are not expected to know parallel computing yet. Work individually and show your reasoning.

Instructions: Use a language you know. Do not worry about perfect syntax unless asked. For algorithm questions, explain your thinking. If you do not know something, say so and move on.

Task 1 — Trace the Code (5 min)

Determine the final output. Show the value of x and y after each statement.

```
x = 5
```

```
y = 2
```

```
x = x + y
```

```
y = x * 2
```

```
x = y - x
```

```
print(x, y)
```

```
x=7
```

```
y=14
```

Task 2 — Find the Bug (5 min)

What does this program actually calculate? What was it probably intended to calculate? Fix it.

```
numbers = [10, 20, 30, 40, 50]
```

```
total = 0
```

```
for i in range(len(numbers)):
    total = numbers[i]
```

```
print(total)
```

```
total=10
```

 In the end the program prints: 50

```
total=20
```

 This program intended to calculate the sum of numbers.

```
total=30
```

```
total=40
```

```
total=50
```

Fix:

```
numbers = [10, 20, 30, 40, 50]
```

```
total = 0
```

```
for i in range(len(numbers)):
    total += numbers[i]
```

```
print(total)
```

Task 3 — Nested Loops (5 min)

For the program below: (a) how many times does `print()` execute? (b) what if both loops run from 0 to 999? (c) what if the outer loop runs 10 times and the inner loop runs 1,000,000 times?

a)100 times b) million times c)10 million tim

```
for i in range(10):
    for j in range(10):
        print(i, j)
```

Part B — Algorithms & Complexity

Task 4 — Which Is Faster? (5 min)

Algorithm A scans an array once. Algorithm B compares every element with every element. For an array of size n , estimate the amount of work for each. Which grows faster? What happens when $n = 1,000,000$?

A grows much more slowly and is much faster for very large n .

Task 5 — Complexity Ranking (7 min)

Rank these from fastest-growing to slowest-growing as n

increases: n^2 $\log_2 n$ n $n \log_2 n$ 1 2^n

Which become particularly problematic when n becomes much larger?

2^n n^2 $n \log_2 n$ $n \log_2 n$ 1

worst as n becomes huge are 2^n and n^2

Especially: 2^n because exponential growth becomes enormous extremely quickly

Task 6 — Searching (7 min)

Given [3, 8, 12, 17, 24, 31, 45, 51, 63], find 45 using two approaches: (1) if you do not know the list is sorted; (2) if you know it is sorted. Explain why the second can be faster.

1) Check one by one 3 → no 8 → no 12 → no 17 → no 24 → no 31 → no 45 → YES

2) Take first middle 24 < 45 so we take right side

Take also middle 45 = 45 so it is found

Linear check one by one so that is slow, binary check dividing to half

So this is faster.

Part C — Data & Problem Solving

Task 7 — Large Dataset (8 min)

You have 500 million integers and need to find the largest number. Design an algorithm. How many times does each number need to be examined? Do you need to store all 500 million numbers? Could you process the data in chunks?

Examined once. No, we only need the current number and current maximum. Yes, because

chunk finds maximum.

Task 8 — Counting (5 min)

Given A B A C B A D C A B, design an algorithm producing $A \rightarrow 4$, $B \rightarrow 3$, $C \rightarrow 2$, $D \rightarrow 1$. What data structure would you use?

Use a dictionary/map. Finally: $A \rightarrow 4$ $B \rightarrow 3$ $C \rightarrow 2$ $D \rightarrow 1$

Task 9 — Matrix Operations (5 min)

$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ and $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$. Calculate $A+B$ and $A \times B$. How many individual multiplication operations are required for two 2×2 matrices?

A + B

Add corresponding elements:

$$1+5 = 6$$

$$2+6 = 8$$

$$3+7 = 10$$

$$4+8 = 12$$

Answer:

$$\begin{bmatrix} 6 & 8 \end{bmatrix},$$

$$\begin{bmatrix} 10 & 12 \end{bmatrix}$$

A × B

Formula:

$$C[i][j] = \text{sum}(A[i][k] \times B[k][j])$$

First element:

$$1 \times 5 + 2 \times 7$$

$$= 5 + 14$$

$$= 19$$

Second:

$$1 \times 6 + 2 \times 8$$

$$= 6 + 16$$

$$= 22$$

Third:

$$3 \times 5 + 4 \times 7$$

= 15 + 28

= 43

Fourth:

$3 \times 6 + 4 \times 8$

= 18 + 32

= 50

Answer:

[[19,22],

[43,50]]

Part D — Systems Intuition

Task 10 — CPU vs Memory (6 min)

A program scans a huge array, but the CPU is extremely fast and the program is still slow. Give three possible reasons.

1. Memory bandwidth

The CPU may be waiting for data to arrive from memory.

2. Cache misses

The data may not fit into CPU cache, so the CPU repeatedly accesses slower memory.

3. Huge amount of data

Even if each operation is very fast, processing a huge array still takes significant time.

The program may be limited by memory bandwidth, cache misses, and the very large amount of data that must be read.

```
for x in huge_array:
    total += x
```

Task 11 — What Happens When You Run a Program? (7 min)

Describe what happens between typing `./program` and seeing the program's output. Use ordinary language.

Include anything you think is relevant.

The operating system receives the command and finds the program. It loads the program from storage into memory and creates a process. The CPU begins executing the program's instructions. The program may read input, perform calculations, and use memory or other resources. Finally, it produces output, which is displayed on the screen.

Task 12 — Four Programs (7 min)

Four programs run simultaneously: A uses lots of CPU; B uses lots of memory; C reads a large file; D mostly waits for user input. Identify the likely bottleneck for each. Can all four run at the same time? What determines CPU time? What changes if there is one CPU core?

A

Bottleneck:

CPU

B

Bottleneck:

Memory

C

Bottleneck:

Disk / I/O

D

Bottleneck:

Waiting for input / I/O

Can all four run at the same time?

Yes, the operating system can run multiple programs concurrently.

What determines CPU time?

The operating system scheduler determines how CPU time is shared between processes.

What changes with one CPU core?

Only one program/thread can execute instructions on that core at a given instant, so programs take turns using the CPU. I/O-waiting programs may not need the CPU while they are waiting.

Part E — Reasoning Challenges

Task 13 — The Slow Program (6 min)

A program processes 1 billion records in 10 hours. A student says: “If we buy a computer that is twice as fast, the program will take 5 hours.” Do you agree? Explain.

Not necessarily.

A faster CPU does not guarantee that the whole program becomes twice as fast.

Why?

The program might be limited by:

- memory
- disk I/O
- network
- synchronization
- another part of the program

If the CPU is the only bottleneck, then approximately:

$$10 / 2 = 5 \text{ hours}$$

But if another resource is the bottleneck, the actual time may be greater than 5 hours.

Task 14 — Make It Faster (6 min)

A program takes 60 seconds. You may change anything except the answer it produces. List as many ways as possible to reduce execution time.

Use a better algorithm.

Reduce unnecessary calculations.

Reduce unnecessary loops.

Use more efficient data structures.

Reduce disk I/O.

Reduce memory usage.

Use caching.

Use multiple CPU cores / parallel processing.

Process data in chunks.

Improve database queries if a database is used.

Avoid repeated calculations.

Use faster hardware if appropriate.

Task 15 — Bottleneck Identification (6 min)

Execution times are: A=2s, B=2s, C=50s, D=3s, E=2s. Total=59s. If A, B, D and E become 10× faster, how much faster does the entire program become? Where should optimization effort go?

New times

A:

$$2 / 10 = 0.2s$$

B:

$$2 / 10 = 0.2s$$

C stays:

50s

D:

$$3 / 10 = 0.3s$$

E:

$$2 / 10 = 0.2s$$

New total:

$$\begin{aligned} &0.2 + 0.2 + 50 + 0.3 + 0.2 \\ &= 50.9 \text{ seconds} \end{aligned}$$

Original:

59 seconds

Speedup:

$$59 / 50.9 \approx 1.16\times$$

So the entire program becomes only about **1.16 times faster**.

Where should optimization effort go?

Task C, because it takes **50 of the 59 seconds**.

That's approximately:

$$50 / 59 \approx 85\%$$

of the total execution time.

This is the main bottleneck.

Final Challenge — Design the Algorithm

Task 16 — 10 Million Student Marks (10 min)

You receive 10 million student marks, each between 0 and 100. You must produce: minimum, maximum, average, median, number scoring ≥ 90 , and number scoring < 40 . Design an algorithm; identify data structures; estimate how many times the data must be examined; estimate memory requirements; identify the likely bottleneck; and explain how you would make it faster without changing the answer.

Step 1 — Minimum and maximum

Keep:

minimum
maximum

Read every mark once.

For every mark:

if mark < minimum $\rightarrow$ update minimum
if mark > maximum $\rightarrow$ update maximum

Step 2 — Average

Keep:

sum
count

For every mark:

sum += mark
count += 1

At the end:

average = sum / count

Step 3 — Count ≥ 90

For every mark:

```
if mark >= 90:  
    count90 += 1
```

Step 4 — Count <40

For every mark:

```
if mark < 40:  
    count40 += 1
```

Step 5 — Median

This is the more difficult part.

Because marks are only between **0 and 100**, we can use a frequency array:

```
frequency[0...100]
```

For each mark:

```
frequency[mark] += 1
```

For example:

```
frequency[90] = number of students who scored 90
```

Then use the frequency counts to determine the middle position(s).

This avoids needing to sort 10 million individual numbers.

Data Structures

Use:

minimum → one variable

maximum → one variable

sum → one variable

count ≥90 → one variable

count <40 → one variable

frequency → array of 101 integers

How many times must the data be examined?

Ideally:

10 million marks

can be processed in **one pass**.

Each mark is examined once.

Then we use the small frequency array to determine the median.

Memory requirements

We **do not need to store all 10 million marks**.

We only need:

101 frequency counters

plus a few variables.

Therefore memory usage is very small compared with storing all 10 million marks.

Likely bottleneck

The likely bottleneck is **reading/processing the 10 million marks**, especially if they come from disk or another external source.

How to make it faster?

Possible approaches:

- process data in chunks;
- use multiple CPU cores;
- divide the 10 million marks between workers;
- calculate partial sums/minimums/maximums/counts;
- combine the partial results;
- use efficient input/output;
- use the frequency array for the median instead of sorting all marks.

For example:

Worker 1 → marks 1–2,500,000

Worker 2 → marks 2,500,001–5,000,000

Worker 3 → marks 5,000,001–7,500,000

Worker 4 → marks 7,500,001–10,000,000

Each worker calculates its own:

min
max
sum
count ≥ 90
count < 40
frequency[0...100]

Then combine all results.

That is where **parallel computing** starts becoming useful.

STOP HERE. Check that you attempted every task. You are not expected to know parallel computing yet.